Student Expense Tracker and Budget Management System

Python Internship — Week 6 Final Project Report

Project Details

Student Name: D. Jeevan Eshwar

Internship Domain: Python Programming & Automation

Week: 6 of 6 Project Title: Student Expense Tracker and Budget Management System

GitHub Repository: https://github.com/jeevaneshwar22-dev/Internova-Internship

1. Project Title

Student Expense Tracker and Budget Management System

2. Introduction

The Student Expense Tracker and Budget Management System is a Python-based, menu-driven application designed to help students record their daily expenses and monitor their monthly budget.

The application allows users to add, view, search, and delete expense records, review monthly spending, generate category-wise expense reports, and export expense records to a CSV file.

The project uses JSON files to store expense records and budget settings so that the data remains available after the application is closed. It demonstrates important Python programming concepts, including functions, data structures, file handling, exception handling, and object-oriented programming.

3. Problem Statement

Students often spend money on food, transportation, education, shopping, and other daily activities. Without proper records, it can be difficult to understand spending habits, calculate total expenses, and determine how much money remains within a monthly budget.

This project aims to provide a simple and organized application that allows students to record their expenses, monitor monthly spending, manage a budget, and generate expense reports.

4. Project Objectives

The main objectives of this project are:

1. To develop a simple Python-based expense management application.
2. To record and maintain daily expense details.
3. To calculate total expenses and monitor monthly spending.
4. To allow users to set and track a monthly budget.
5. To provide search and deletion facilities for expense records.
6. To store information persistently using JSON files.
7. To generate category-wise expense reports.
8. To export expense records into CSV format.
9. To apply Python programming concepts in a practical project.
10. To gain experience in debugging, testing, and problem-solving.
11. To demonstrate version control and project hosting using Git and GitHub.

5. Features of the Project

5.1 Add Expense

Users can enter an expense amount, category, description, and date. The application validates the input and saves the expense record.

5.2 View All Expenses

The application displays the recorded expenses, including their IDs, dates, categories, amounts, and descriptions.

5.3 Search Expenses

Users can search for expenses using keywords in the category or description. This makes it easier to find specific records.

5.4 Delete Expense

Users can delete an expense by entering its unique expense ID.

5.5 Set Monthly Budget

Users can configure a monthly spending limit. The application stores the budget setting for future sessions.

5.6 View Monthly Budget

The application calculates the total expenses for the current month and displays the monthly budget, amount spent, and remaining budget.

5.7 Generate Expense Report

The application generates a report showing category-wise expense totals and overall expenditure.

5.8 Export Report to CSV

Users can export expense records into a CSV file. This file can be opened using spreadsheet applications such as Microsoft Excel or Google Sheets.

5.9 Data Persistence

Expense records and budget settings are stored in JSON files. This allows the application to retrieve previously saved information when it is started again.

5.10 Input Validation and Error Handling

The application handles common invalid inputs and file-related errors to improve reliability and provide useful feedback to users.

6. Technologies Used

Technology	Purpose
Python 3	Main programming language
`json` module	Storing and retrieving expense records and budget settings
`csv` module	Exporting expense records into CSV format
`datetime` module	Working with dates and filtering expenses by month
`pathlib` module	Managing file paths
`abc` module	Defining abstract classes and methods
Visual Studio Code / Python IDLE	Writing and executing Python code
Git	Tracking code changes and maintaining version history
GitHub	Hosting and sharing the project repository

The project uses Python's standard library and does not require third-party packages.

7. Python Concepts Implemented

7.1 Variables and Data Types

Variables are used to store expense amounts, categories, descriptions, dates, expense IDs, and budget values.

The application uses data types such as strings, integers, floating-point numbers, lists, and dictionaries.

7.2 Input and Output

The `input()` function collects information from the user, while the `print()` function displays menus, expense records, reports, and messages.

7.3 Operators and Type Casting

Arithmetic operators are used to calculate total expenses and the remaining budget. User input is converted into appropriate numeric types when required.

7.4 Conditional Statements

The `if`, `elif`, and `else` statements are used to process menu choices, validate input, and check budget conditions.

7.5 Loops

A `while` loop keeps the application running until the user selects the exit option.

`for` loops are used to process expense records, search for matching expenses, and calculate totals.

7.6 Functions and Methods

Functions and methods divide the application into smaller operations, including adding, viewing, searching, deleting, saving, and reporting expenses.

This improves code organization and makes the program easier to maintain.

7.7 Data Structures

Lists are used to maintain expense records, while dictionaries represent expense data and category-wise totals.

These data structures help organize and process information efficiently.

7.8 File Handling

The application uses JSON files to store expense records and budget settings. It also uses CSV file handling to export expense records into a spreadsheet-compatible format.

7.9 Exception Handling

`try` and `except` blocks are used to handle common errors, such as invalid user input and file or JSON-related problems.

Exception handling prevents certain errors from terminating the application unexpectedly.

7.10 Python Standard-Library Modules

The project uses the following modules:

- `json` for data storage.
- `csv` for report export.
- `datetime` for date handling.
- `pathlib` for file paths.
- `abc` for abstract classes and methods.

These modules provide useful functionality without requiring additional package installation.

7.11 Classes and Objects

The project uses object-oriented programming to organize the application.

- Expense: Represents an individual expense record.
- ExpenseTracker: Manages expense records, budget settings, and application operations.
- Report: Defines the structure for report generation.
- ConsoleReport: Generates and displays expense reports in the console.
- BaseTracker: Defines the abstract operations required by the expense tracker.

7.12 Constructors

The `__init__()` method initializes objects and their attributes.

For example, the `Expense` class initializes the details of an expense, while the tracker initializes its internal data and budget state.

7.13 Inheritance

Inheritance allows a class to reuse or extend the functionality of another class.

In this project, `ExpenseTracker` inherits from the abstract `BaseTracker` class, and `ConsoleReport` inherits from the abstract `Report` class.

This structure helps organize related functionality and separate responsibilities.

7.14 Polymorphism

Polymorphism allows different classes to provide their own implementations of a common method.

In this project, the abstract `generate()` method is declared in the `Report` class and implemented by `ConsoleReport`.

CSV export is handled separately through the `ExpenseTracker.export_csv()` method. The current implementation does not contain a separate `CSVReport` class.

7.15 Encapsulation

Encapsulation involves keeping an object's internal data controlled through its methods.

The `ExpenseTracker` class uses private attributes for its internal expense list and monthly budget. Methods manage access to and modification of this data.

The `get_expenses()` method returns a copy of the expense list, helping protect the tracker's internal list from direct modification by callers.

7.16 Abstraction

Abstraction hides implementation details and defines the operations that a class must provide.

The `BaseTracker` and `Report` abstract classes define required operations, while concrete classes implement those operations.

7.17 Git and GitHub

Git is used to track changes to the project files and maintain version history.

GitHub hosts the repository online, allowing the source code, documentation, and supporting files to be shared and reviewed.

8. Project Workflow

The application follows the workflow below:

1. The user starts the application by running `main.py`.
2. The application loads previously saved expense records and budget settings from JSON files, if available.
3. The main menu is displayed.
4. The user selects an operation.
5. The application accepts and validates the required input.
6. The selected operation is performed, such as adding, viewing, searching, or deleting an expense.
7. Updated expense records or budget settings are saved when necessary.
8. The application displays the result, report, or export status.
9. The menu appears again until the user selects Exit.

Workflow summary:

Start Application → Load Saved Data → Display Menu → Select Operation → Validate Input → Process Data → Save Changes → Display Results or Export CSV → Return to Menu → Exit

9. Screenshots and Output

Insert actual screenshots from the tested application in this section. Each screenshot should show the relevant feature working in the submitted version of the program.

9.1 Main Menu

Insert Screenshot 1 here: Show the main menu containing all nine options in the corrected program.

9.2 Adding an Expense

Insert Screenshot 2 here: Show an expense being entered and the corresponding success message.

9.3 Viewing All Expenses

Insert Screenshot 3 here: Show expense IDs, dates, categories, amounts, and descriptions.

9.4 Searching Expenses

Insert Screenshot 4 here: Show a search operation and the matching expense records.

9.5 Monthly Budget Management

Insert Screenshot 5 here: Show the monthly budget, the amount spent during the current month, and the remaining budget.

9.6 Generating an Expense Report

Insert Screenshot 6 here: Show the category-wise expense totals and overall expenditure.

9.7 Exporting the Report to CSV

Insert Screenshot 7 here: Show the successful CSV export message. If possible, also show the generated CSV file containing the expense records.

9.8 Git Workflow

Insert Screenshot 8 here: Show genuine evidence of the Git workflow, including commands such as `git --version`, `git init`, `git add`, `git commit`, and a successful `git push`.

9.9 GitHub Repository

Insert Screenshot 9 here: Show the GitHub repository with the uploaded source code and documentation.

Sample Expense Report Output

The following is an illustrative example of how a category-wise expense report may appear. These values are examples, not verified output from a test run.

========== EXPENSE REPORT ==========

Food: Rs. 500.00

Transport: Rs. 200.00
Utilities: Rs. 300.00

Total expenses: Rs. 1000.00

Actual output depends on the expense records entered by the user.

10. Challenges Faced

10.1 Handling Invalid Input

Expense amounts and dates must be entered in valid formats. Input validation and exception handling help manage common invalid entries.

10.2 Maintaining Data Permanently

The application needs to preserve expense records and budget settings even after it is closed. JSON files are used to store and retrieve this information.

10.3 Managing Expense Records

Adding, displaying, searching, and deleting expense records requires a structured way to identify each record. Unique expense IDs help users select the correct record.

10.4 Calculating Monthly Expenses

The application must distinguish current-month expenses from expenses recorded during other months. Date filtering is used to calculate the appropriate monthly total.

10.5 Organizing Object-Oriented Code

Separating expense records, tracker operations, and report generation into classes helps improve code structure and maintainability.

10.6 Generating Reports

The application processes expense records to calculate category-wise totals and overall expenditure. CSV export also requires writing the data in a suitable tabular format.

10.7 Debugging and Testing

Testing normal and invalid inputs is important for identifying problems with menu choices, data files, calculations, and user input.

The application should be tested with different scenarios to ensure that its features work as expected.

11. Future Improvements

The project can be enhanced in several ways:

1. Graphical User Interface: Develop a desktop interface using Tkinter.
2. Web Application: Build a browser-based version using Flask or Django.
3. Database Integration: Replace JSON storage with SQLite or another database as the application grows.
4. User Authentication: Introduce individual accounts if multi-user functionality is required.
5. Visual Analytics: Display spending patterns using charts and graphs.
6. Budget Alerts: Notify users when spending approaches or exceeds their monthly budget.
7. Edit Expense Feature: Allow users to modify existing expense records.
8. Recurring Expenses: Support recurring subscriptions and monthly bills.
9. PDF Reports: Generate downloadable PDF summaries.
10. Automated Testing: Add unit tests for core features and edge cases.
11. Cloud Backup: Introduce secure backup and synchronization when needed.
12. Improved Security: Add suitable access controls and data protection if the application is expanded to support multiple users.

12. Conclusion

The Student Expense Tracker and Budget Management System is a Python-based application designed to simplify expense recording and monthly budget monitoring.

Its intended functionality includes adding, viewing, searching, and deleting expense records, monitoring current-month spending, generating category-wise reports, and exporting expense records to CSV format.

The project demonstrates Python fundamentals, data structures, functions, file handling, exception handling, standard-library modules, and object-oriented programming concepts such as inheritance, encapsulation, abstraction, and polymorphism.

The project also provides practical experience in organizing code, validating input, managing local data, debugging, documenting software, and using Git and GitHub for version control and repository hosting.

Future enhancements, such as database integration, a graphical interface, automated testing, and visual analytics, could make the application more useful and scalable.

Before submission, the report, screenshots, README, and source code should be checked against the same tested version of the application to ensure consistency and accuracy.

Submission Details

Student Name: D. Jeevan Eshwar Internship Domain: Python Programming & Automation Internship Week: Week 6 of 6 Project Title: Student Expense Tracker and Budget Management System GitHub Repository: https://github.com/jeevaneshwar22-dev/Internova-Internship